

Project Design Phase-II — Technology Stack (Architecture & Stack)

Date	8 July 2026
Team ID	SmartBridge Externship
Project Name	FinRelief AI
Maximum Marks	4 Marks

Table 1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Responsive single-page application: login/register, dashboard, loan management, settlement simulation, letter generation	React 19 + Vite 8, Tailwind CSS v4
2	Charts	Stress trend and DTI visualisations on the dashboard	Recharts
3	API Client	Authenticated HTTP calls from frontend to backend	Axios
4	Application Logic	REST API: authentication, loan CRUD, settlement calculation, letter generation, dashboard aggregation	FastAPI (Python)
5	ORM	Maps Python model classes to database tables	SQLAlchemy 2.0
6	Cloud Database	Persistent relational store for users, loans, settlements, and letters	Neon serverless PostgreSQL

S.No	Component	Description	Technology
7	Test Database	Isolated, disposable database used only during automated tests	SQLite
8	External API	Drafts the OTS negotiation letter body	Google Gemini 2.0 Flash
9	File Generation	Renders the negotiation letter as a downloadable PDF	ReportLab
10	Infrastructure (Cloud)	Backend hosting: Render. Frontend hosting: Vercel	Render, Vercel

Table 2: Application Characteristics

S.No	Characteristics	Description	Technology Used
1	Open-Source Frameworks	React, FastAPI, SQLAlchemy, Recharts, Axios, ReportLab	Listed frameworks above
2	Security Implementations	Bcrypt password hashing, JWT-based stateless sessions, SlowAPI rate limiting on sensitive endpoints, secrets loaded only from environment variables	bcrypt, python-jose (JWT), SlowAPI
3	Scalable Architecture	Fully decoupled frontend/backend communicating only via REST/JSON, deployed as two independently scalable services	React + FastAPI, decoupled deployment
4	Availability	Google Gemini calls have an automatic fallback to a static professional template if the AI service fails or times out, so the	Gemini + fallback template pattern

S.No	Characteristics	Description	Technology Used
		letter-generation feature never breaks	
5	Performance	Settlement calculations are synchronous, pure functions with no external calls; only the AI letter-generation call has any meaningful external latency	FastAPI synchronous computation

Notes

Neon PostgreSQL was chosen over a locally hosted database specifically because Render and Vercel use ephemeral filesystems that are wiped on every redeploy — a file-based database would lose data on every deploy. Neon persists independently of the application's own deploy lifecycle. SQLite is used only for the isolated automated test suite, never for production data.